ASSIGNMENT : 12.2

Name : J.Akshaya

Bt.no : 22

Ht.no : 2303A51629

Task Description -1 (Data Structures – Stack Implementation with AI Assistance)

➢ Task: Use AI assistance to generate a Python program that implements a Stack data structure.

Instructions:

Prompt AI to create a Stack class with the following methods:

➢ push(element)

➢ pop()

➢ peek()

➢ is_empty()

➢ Ensure proper error handling for stack underflow.

➢ Ask AI to include clear docstrings for each method.

Expected Output:

➢ A functional Python program implementing a Stack using a class.

➢ Properly documented methods with docstrings.

Output screenshot:

assignment08.py | 17-02-2026.py | ass9.2.py | assignment11.py | as12.py | ass

as12.py > ...

```python
# ➢ Task: Use AI assistance to generate a Python program that implements a Stack data
# Prompt AI to create a Stack class with the following methods:
# ➢ push(element)   ➢ pop()
# ➢ peek()  ➢ is_empty()
# ➢ Ensure proper error handling for stack underflow.
# ➢ Ask AI to include clear docstrings for each method. Expected Output:
# ➢ A functional Python program implementing a Stack using a class.
# ➢ Properly documented methods with docstrings.
class Stack:
    """
    A class to represent a Stack data structure.
    """

    def __init__(self):
        """
        Initializes an empty stack.
        """
        self.stack = []

    def push(self, element):
        """
        Adds an element to the top of the stack.

        Parameters:
        element (any): The element to be added to the stack.
        """
        self.stack.append(element)

    def pop(self):
        """
        Removes and returns the top element of the stack.

        Returns:
        any: The element removed from the top of the stack.

        Raises:
```

s12.py > ...

```python
class Stack:
    def pop(self):
        """
        Raises:
        IndexError: If the stack is empty (stack underflow).
        """
        if self.is_empty():
            raise IndexError("Stack underflow: Cannot pop from an empty stack.")
        return self.stack.pop()

    def peek(self):
        """
        Returns the top element of the stack without removing it.

        Returns:
        any: The top element of the stack.

        Raises:
        IndexError: If the stack is empty (stack underflow).
        """
        if self.is_empty():
            raise IndexError("Stack underflow: Cannot peek from an empty stack.")
        return self.stack[-1]

    def is_empty(self):
        """
        Checks if the stack is empty.

        Returns:
        bool: True if the stack is empty, False otherwise.
        """
        return len(self.stack) == 0
# Example usage:
```

```python
        Returns:
        bool: True if the stack is empty, False otherwise.
        """
        return len(self.stack) == 0
# Example usage:
if __name__ == "__main__":
    stack = Stack()
    stack.push(1)
    stack.push(2)
    stack.push(3)
    print(stack.peek())  # Output: 3
    print(stack.pop())   # Output: 3
    print(stack.is_empty())  # Output: False
    print(stack.pop())   # Output: 2
    print(stack.pop())   # Output: 1
    print(stack.is_empty())  # Output: True
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                Python

PS C:\Users\aksha\OneDrive\Desktop\assisantcoding - Copy> & C:/Users/aksha/AppData/Local/Microsoft/WindowsApp
s/python3.13.exe "c:/Users/aksha/OneDrive/Desktop/assisantcoding - Copy/as12.py"
3
3
False
2
1
True
PS C:\Users\aksha\OneDrive\Desktop\assisantcoding - Copy> & C:/Users/aksha/AppData/Local/Microsoft/WindowsApp
s/python3.13.exe "c:/Users/aksha/OneDrive/Desktop/assisantcoding - Copy/as12.py"
```

Task Description -2 (Algorithms – Linear vs Binary Search Analysis)

➢ Task: Use AI to implement and compare Linear Search and Binary Search algorithms in Python.

Instructions:

➢ Prompt AI to generate:

➢ linear_search(arr, target)

➢ binary_search(arr, target)

➢ Include docstrings explaining:

➢ Working principle

➢ Test both algorithms using different input sizes.

Expected Output:

• Python implementations of both search algorithms.

• AI-generated comments and complexity analysis.

• Test results showing correctness and comparison.

ssignment08.py | 17-02-2026.py | ass9.2.py | assignment11.py | as12.py | ass

as12.py > ...

```python
# ➢ Task: Use AI to implement and compare Linear Search and Binary Search algorithms
# ➢ Prompt AI to generate:  ➢ linear_search(arr, target)
# ➢ binary_search(arr, target)  ➢ Include docstrings explaining:
# ➢ Working principle   ➢ Test both algorithms using different input sizes. Expected
# • Python implementations of both search algorithms.
# • AI-generated comments and complexity analysis.
# • Test results showing correctness and comparison.
def linear_search(arr, target):
    """
    Performs a linear search for the target element in the given array.

    Parameters:
    arr (list): The list in which to search for the target element.
    target (any): The element to search for.

    Returns:
    int: The index of the target element if found, otherwise -1.

    Complexity:
    Time Complexity: O(n) - In the worst case, it checks each element once.
    Space Complexity: O(1) - No additional space is used.
    """
    for index in range(len(arr)):
        if arr[index] == target:
            return index
    return -1
def binary_search(arr, target):
    """
    Performs a binary search for the target element in the given sorted array.

    Parameters:
    arr (list): The sorted list in which to search for the target element.
    target (any): The element to search for.

    Returns:
```

```python
    Parameters:
    arr (list): The sorted list in which to search for the target element.
    target (any): The element to search for.

    Returns:
    int: The index of the target element if found, otherwise -1.

    Complexity:
    Time Complexity: O(log n) - In the worst case, it halves the search space with eac
    Space Complexity: O(1) - No additional space is used.
    """
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = left + (right - left) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
# Example usage:
if __name__ == "__main__":
    arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
    target = 5

    # Testing Linear Search
    linear_result = linear_search(arr, target)
    print(f"Linear Search: Target {target} found at index {linear_result}" if linear_r

    # Testing Binary Search
    binary_result = binary_search(arr, target)
    print(f"Binary Search: Target {target} found at index {binary_result}" if binary_r
```

Output:

```
True
PS C:\Users\aksha\OneDrive\Desktop\assisantcoding - Copy> & C:/Users/aksha/AppData/Local/Microsoft/WindowsAp
s/python3.13.exe "c:/Users/aksha/OneDrive/Desktop/assisantcoding - Copy/as12.py"
Linear Search: Target 5 found at index 4
Binary Search: Target 5 found at index 4
PS C:\Users\aksha\OneDrive\Desktop\assisantcoding - Copy>
```

Task Description -3 (Test Driven Development – Simple Calculator Function)

➢ Task:

Apply Test Driven Development (TDD) using AI assistance to develop a calculator function.

Instructions:

➢ Prompt AI to first generate unit test cases for addition and subtraction.

➢ Run the tests and observe failures.

➢ Ask AI to implement the calculator functions to pass all tests.

➢ Re-run the tests to confirm success.

Expected Output:

➢ Separate test file and implementation file. ➢ Test cases executed before implementation.

➢ Final implementation passing all test cases.

Code screenshot:

```
> # Task Description -3 (Test Driven Development – Simple Calculator Function) ···
  # Apply Test Driven Development (TDD) using AI assistance to develop a calculator func

  # Instructions:
  # ➢ Prompt AI to first generate unit test cases for addition and subtraction.
  # ➢ Run the tests and observe failures.
  # ➢ Ask AI to implement the calculator functions to pass all tests.
  # ➢ Re-run the tests to confirm success. Expected Output:
  # ➢ Separate test file and implementation file. ➢ Test cases executed before implemer
  # ➢ Final implementation passing all test cases.

  # test_calculator.py
  import unittest
  from calculator import add, subtract

  class TestCalculator(unittest.TestCase):
      def test_add(self):
          self.assertEqual(add(2, 3), 5)
          self.assertEqual(add(-1, 1), 0)
          self.assertEqual(add(0, 0), 0)

      def test_subtract(self):
          self.assertEqual(subtract(5, 2), 3)
          self.assertEqual(subtract(0, 1), -1)
          self.assertEqual(subtract(-1, -1), 0)
  if __name__ == '__main__':
      unittest.main()
  # calculator.py
  def add(a, b):
      """
      Adds two numbers.
```

➢

```python
        self.assertEqual(subtract(-1, -1), 0)
if __name__ == '__main__':
    unittest.main()
# calculator.py
def add(a, b):
    """
    Adds two numbers.

    Parameters:
    a (int): The first number.
    b (int): The second number.

    Returns:
    int: The sum of a and b.
    """
    return a + b
def subtract(a, b):
    """
    Subtracts the second number from the first number.

    Parameters:
    a (int): The first number.
    b (int): The second number.

    Returns:
    int: The difference of a and b.
    """
    return a - b
```

Task Description -4 (Data Structures – Queue Implementation with AI Assistance)

Use AI assistance to generate a Python program that implements a Queue data structure.

Instructions:

➢ Prompt AI to create a Queue class with the following methods:

- enqueue(element)
- dequeue()
- front()
- is_empty()

➢ Handle queue overflow and underflow conditions.

➢ Include appropriate docstrings for all methods.

Expected Output:

➢ A fully functional Queue implementation in Python.

Code :

```python
# Task Description -4 (Data Structures – Queue Implementation with AI Assistance)
# Use AI assistance to generate a Python program that implements a Queue data structur
# ➢ Prompt AI to create a Queue class with the following methods:
# • enqueue(element)    • dequeue()
# • front() • is_empty()
# ➢ Handle queue overflow and underflow conditions.
# ➢ Include appropriate docstrings for all methods. Expected Output:
# ➢ A fully functional Queue implementation in Python.

class Queue:
    """
    A class to represent a Queue data structure.
    """

    def __init__(self):
        """
        Initializes an empty queue.
        """
        self.queue = []

    def enqueue(self, element):
        """
        Adds an element to the end of the queue.

        Parameters:
        element (any): The element to be added to the queue.
        """
        self.queue.append(element)

    def dequeue(self):
        """
        Removes and returns the front element of the queue.

        Returns:
        any: The element removed from the front of the queue.
```

```python
    def dequeue(self):
        """
        Removes and returns the front element of the queue.

        Returns:
        any: The element removed from the front of the queue.

        Raises:
        IndexError: If the queue is empty (queue underflow).
        """
        if self.is_empty():
            raise IndexError("Queue underflow: Cannot dequeue from an empty queue.")
        return self.queue.pop(0)

    def front(self):
        """
        Returns the front element of the queue without removing it.

        Returns:
        any: The front element of the queue.

        Raises:
        IndexError: If the queue is empty (queue underflow).
        """
        if self.is_empty():
            raise IndexError("Queue underflow: Cannot access front of an empty queue."
        return self.queue[0]

    def is_empty(self):
        """
        Checks if the queue is empty.
```

```python
        if self.is_empty():
            raise IndexError("Queue underflow: Cannot access front of an empty queue."
        return self.queue[0]

    def is_empty(self):
        """
        Checks if the queue is empty.

        Returns:
        bool: True if the queue is empty, False otherwise.
        """
        return len(self.queue) == 0
# Example usage:
if __name__ == "__main__":
    queue = Queue()
    queue.enqueue(1)
    queue.enqueue(2)
    queue.enqueue(3)
    print(queue.front())  # Output: 1
    print(queue.dequeue())  # Output: 1
    print(queue.is_empty())  # Output: False
    print(queue.dequeue())  # Output: 2
    print(queue.dequeue())  # Output: 3
    print(queue.is_empty())  # Output: True
```

Output:

```
PS C:\Users\aksha\OneDrive\Desktop\assisantcoding - Copy> & C:/Users/aksha/AppData/Local/Microsoft/WindowsApp
s/python3.13.exe "c:/Users/aksha/OneDrive/Desktop/assisantcoding - Copy/as12.py"
1
1
False
2
3
True
PS C:\Users\aksha\OneDrive\Desktop\assisantcoding - Copy>
```

Task Description -5 (Algorithms – Bubble Sort vs Selection Sort)

➢ Task:

Use AI to implement Bubble Sort and Selection Sort algorithms

and compare their behavior.

Instructions:

➢ Prompt AI to generate:

• bubble_sort(arr) • selection_sort(arr)

➢ Include comments explaining each step.

➢ Add docstrings mentioning time and space complexity.

Expected Output:

• Correct Python implementations of both sorting algorithms.

• Complexity analysis in docstrings.

Code output:

```python
# Task Description -5 (Algorithms – Bubble Sort vs Selection Sort)
# ➢ Task:
# Use AI to implement Bubble Sort and Selection Sort algorithms and compare their beha
# Instructions:
# ➢ Prompt AI to generate:
# • bubble_sort(arr)    • selection_sort(arr)
# ➢ Include comments explaining each step.
# ➢ Add docstrings mentioning time and space complexity. Expected Output:
# • Correct Python implementations of both sorting algorithms.
# • Complexity analysis in docstrings.
def bubble_sort(arr):
    """
    Sorts an array using the Bubble Sort algorithm.

    Parameters:
    arr (list): The list to be sorted.

    Returns:
    list: The sorted list.

    Complexity:
    Time Complexity: O(n^2) - In the worst case, it compares each element with every o
    Space Complexity: O(1) - No additional space is used.
    """
    n = len(arr)
    for i in range(n):
        # Flag to check if any swapping occurs
        swapped = False
        for j in range(0, n-i-1):
            # Compare adjacent elements and swap if they are in the wrong order
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
                swapped = True
        # If no swapping occurred, the array is already sorted
```

```python
def bubble_sort(arr):
    for i in range(n):
        # Flag to check if any swapping occurs
        swapped = False
        for j in range(0, n-i-1):
            # Compare adjacent elements and swap if they are in the wrong order
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
                swapped = True
        # If no swapping occurred, the array is already sorted
        if not swapped:
            break
    return arr
def selection_sort(arr):
    """
    Sorts an array using the Selection Sort algorithm.

    Parameters:
    arr (list): The list to be sorted.

    Returns:
    list: The sorted list.

    Complexity:
    Time Complexity: O(n^2) - In the worst case, it finds the minimum element and swap
    Space Complexity: O(1) - No additional space is used.
    """
    n = len(arr)
    for i in range(n):
        # Find the index of the minimum element in the remaining unsorted portion
        min_index = i
        for j in range(i+1, n):
            if arr[j] < arr[min_index]:
                min_index = j
        # Swap the minimum element with the first element of the unsorted portion
        arr[i], arr[min_index] = arr[min_index], arr[i]
    return arr
```

```python
    n = len(arr)
    for i in range(n):
        # Find the index of the minimum element in the remaining unsorted portion
        min_index = i
        for j in range(i+1, n):
            if arr[j] < arr[min_index]:
                min_index = j
        # Swap the minimum element with the first element of the unsorted portion
        arr[i], arr[min_index] = arr[min_index], arr[i]
    return arr
 Example usage:
f __name__ == "__main__":
    arr1 = [64, 34, 25, 12, 22, 11, 90]
    arr2 = [64, 34, 25, 12, 22, 11, 90]

    print("Original array:", arr1)
    print("Sorted array using Bubble Sort:", bubble_sort(arr1))
    print("Sorted array using Selection Sort:", selection_sort(arr2))
```

Output:

```
PS C:\Users\aksha\OneDrive\Desktop\assisantcoding - Copy> & C:/Users/aksha/AppData/Local/Microsoft/WindowsApp
s/python3.13.exe "c:/Users/aksha/OneDrive/Desktop/assisantcoding - Copy/as12.py"
Original array: [64, 34, 25, 12, 22, 11, 90]
Sorted array using Bubble Sort: [11, 12, 22, 25, 34, 64, 90]
Sorted array using Selection Sort: [11, 12, 22, 25, 34, 64, 90]
PS C:\Users\aksha\OneDrive\Desktop\assisantcoding - Copy>
```